



Architecture et Génie des Logiciels

THÈME : LA FABRICATION D'UN LOGICIEL POUR LA GESTION DES RECRUTEMENTS DU PERSONNEL DE L'UASZ

L3 en Ingénieurie Informatique (L3 - L2I)

Pr. Ibrahima DIOP & Mme Ana BAKHOUM DIANGAR

Logiciel – Définition 1

- Ensemble des programmes, …, relatifs au fonctionnement d'un ensemble de traitement de données ».
- Il s'agit
  - d'un ensemble plus ou moins complexe d'instructions,
  - écrites dans un langage de programmation
  - ayant vocation à être exécutées par un processeur (un ordinateur, un terminal mobile téléphone, tablette, etc.)
  - Pour satisfaire les besoins d'un client

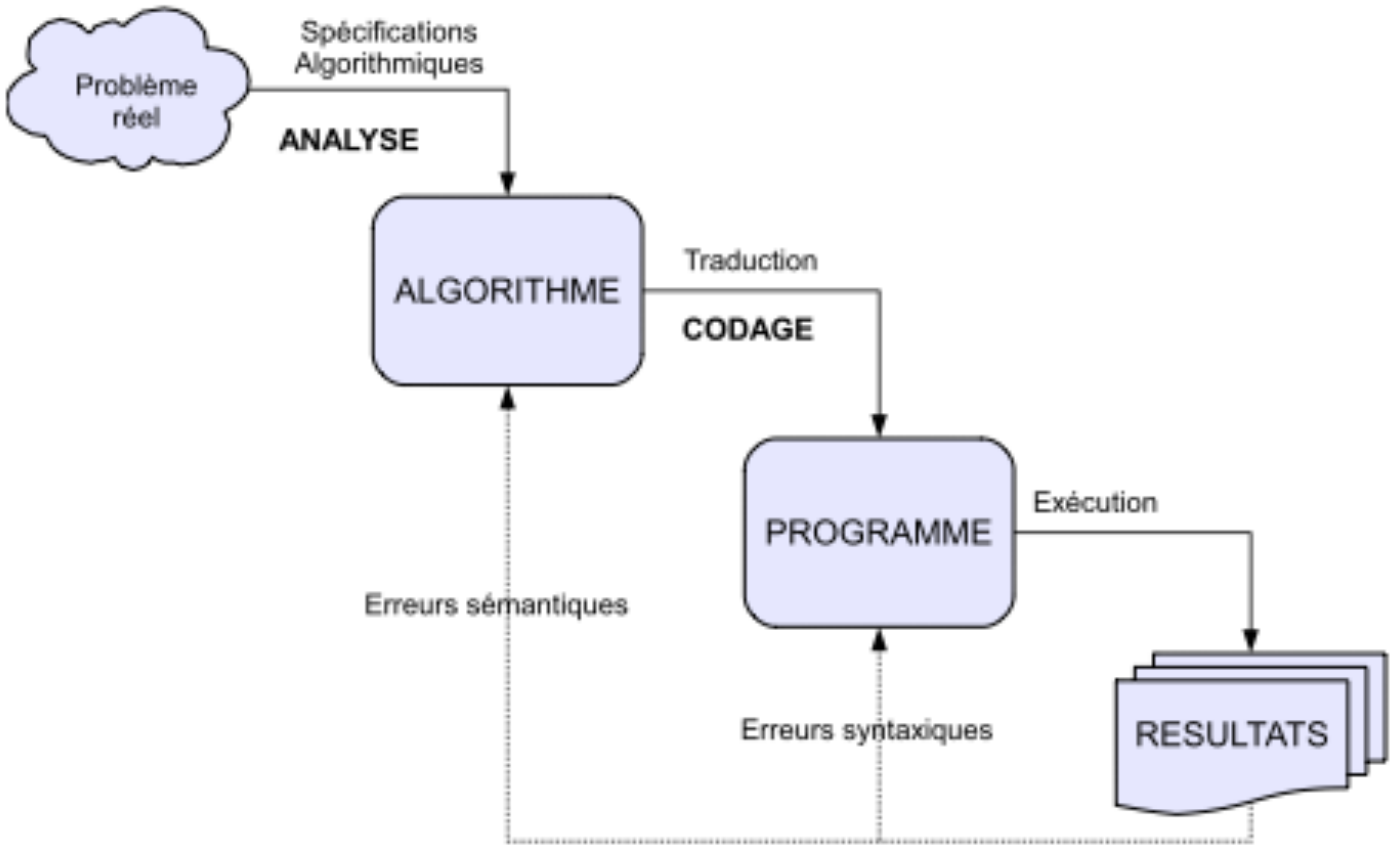
Propriétés du logiciels

- **sa virtualité,** liées à l'essence et à la nature même du logiciel :.
- Monolithique, écrit en un seul bloc
- les **propriétés accidentelles** du logiciel, liées aux problèmes qui entourent la production du logiciel, par exemple les besoins des utilisateurs, qui changent sans cesse ;
- la demande de **fonctionnalités sophistiquées**,
- les **courts délais** de livraison,
- l'inexpérience des développeurs,
- etc.

Langages de programmations des années 70

Années	Langage	Inspiration	Promoteur
1970	**POP-2**	POP-1	
1971	**Pascal**	ALGOL60	Niklaus Wirth, Jensen
1972	**Smalltalk Environment**	Simula67	Digitalk
1972	**C**	B, BCPL	Dennis Ritchie
1972	**INTERCAL**		
1972	**Prolog**		Alain Colmerauer
1972	**SmallTalk 72**		
1973	**COMAL**	Pascal, BASIC	Børge Christensen, Benedict Løfstedt
1974	**GRASS**	BASIC	Tom DeFanti
1974	**COBOL74**	COBOL	
1974	**SQL**	SEQUEL	
1975	**Scheme**	Lisp	Gerald Jay Sussman, Guy Lewis Steele
1975	**Altair BASIC**	BASIC	Bill Gates, Paul Allen
1976	**RATFOR**	C, FORTRAN	Kernighan
1977	**X11.1 ANSI Standard MUMPS**	MUMPS	
1978	**FORTRAN 77**	FORTRAN 66	
1978	**VisiCalc**		Dan Bricklin, Bob Frankston
1979	**REXX**	PL/I	Mike Cowlishaw 20 mars
1979	**Awk**	C, SNOBOL	Alfred Aho, Peter Weinberger, Brian Kernighan
1979	**VULCAN DBase-II**		Ratliff
1979	**Green**	Algol68	Jean Ichbiah *et al.*, DOD

Processus classique de fabrication du logiciel



Exemple d'un logiciel

```
Function addition (val1, val2 : real);
BEGIN
addition := val1 + val2 ;
END.
```

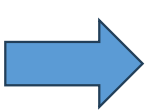
```
Function multiplication (val1, val2 : real);
BEGIN
multiplication := val1 * val2 ;
END.
```

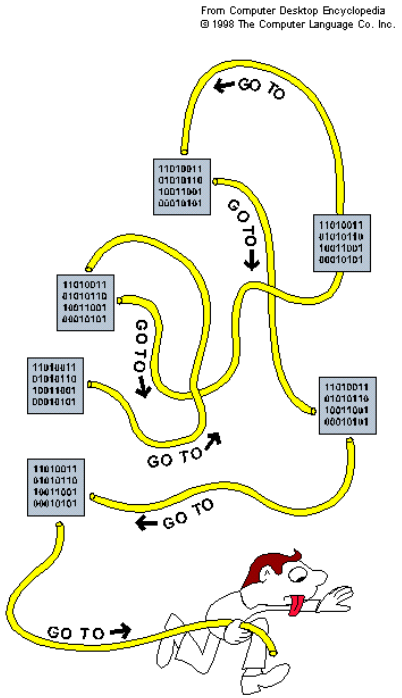
```
Function soustraction (val1, val2 : real);
BEGIN
soustraction := val1 - val2 ;
END.
```

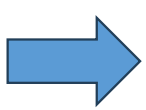
```
Function division (val1, val2 : real);
BEGIN
division := val1 / val2 ;
END.
```

```
PROGRAM calculatrice ;
VAR val1, val2, resultat : real;
      operation : char;
FUNCTION …
BEGIN
   writeln('Première valeur ?');
   readln(val1);
   writeln('Opération (+ - * /) ? ');
   readln(operation);
   writeln('Deuxième valeur ? ');
   readln(val2);
   case operation of
      '+' : resultat := addition(val1 , val2);
      '-' : resultat := soustraction(val1 , val2);
      '*' : resultat := multiplication(val1 , val2);
      '/' : resultat := division(val1 , val2);
  end;
  writeln('Résultat : ', resultat) ;
END.
```

```
PROGRAM calculatrice ;
VAR val1, val2, resultat : real;
        operation : char;
FUNCTION …
BEGIN
    writeln('Première valeur ?');
    readln(val1);
    writeln('Opération (+ - * /) ? ');
    readln(operation);
    writeln('Deuxième valeur ? ');
    readln(val2);
    case operation of
        '+' : resultat := addition(val1 , val2);
        '-' : resultat := soustraction(val1 , val2);
        '*' : resultat := multiplication(val1 , val2);
        '/' : resultat := division(val1 , val2);
   end;
   writeln('Résultat : ', resultat) ;
END.
```









La crise du logiciel

- désigne la situation générale qui caractérisa les échecs de développements logiciels dans les années soixante et soixante-dix.
- Dans son essence le terme fait référence à la difficulté d'écrire des programmes informatiques corrects, compréhensibles et vérifiables.
- On retiendra trois principales causes à l'origine de ce phénomène :
  - la complexité du logiciel
  - les attentes trop importantes des utilisateurs et des clients ;
  - les changements à apporter aux produits logiciels.



La naissance du Génie Logiciel

- Pour résoudre la crise du logiciel, il fallait :
  - comprendre tous les phénomènes sociotechniques observés autour de la production de logiciels
  - et proposer une technique industrielle de production de logiciels s'appuyant sur la connaissance de ces phénomènes.
- Il fallait donc une science et une technique de la production du logiciel : le GL est né.

Qu’est ce que le GL ?

- ***Génie logiciel*** « Software Engineering » est l'application d'une approche systématique, disciplinée et quantifiable au développement, à l'exploitation et à la maintenance des logiciels.
- Il s'agit de l'étude des principes et des pratiques visant à produire des logiciels de haute qualité,
- répondant aux besoins des utilisateurs, tout en étant développés dans les délais et les budgets prévus.

Historique (0)

- **1940-1950** : L'informatique naît avec le développement des premiers ordinateurs électroniques ;
- **1950-1960** : L'avènement des langages de programmation
  - Des langages de programmation plus abstraits et conviviaux, tels que **Fortran et COBOL**, sont développés pour faciliter la programmation.
  - Cela marque aussi la période de la **crise du logiciel**
  - et le début d'une **réflexion sur la manière de concevoir des logiciels** de manière plus méthodique.

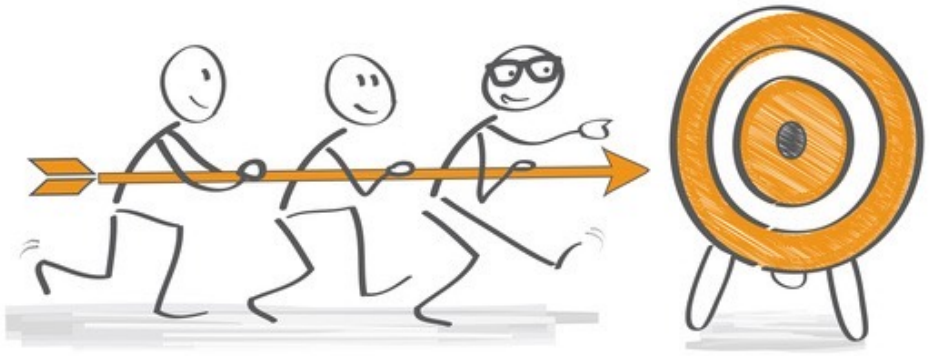
Historique (1)

- 1968 : naissance GL à la conférence de l'OTAN à Garmisch-Partenkirchen (Allemagne)
- 1973 : première conférence sur le GL
- 1975 : première revue sur le GL (IEEE Transaction of Software Engineering)
- 1980 : début des AGL

Historique (2)

- **Les années 1990** : Popularité des méthodologies agiles, telles que Scrum et XP (Extreme Programming).
- **Les années 2000 à nos jours** : Domination des méthodologies agiles, avec une adoption des pratiques comme DevOps, qui favorisent l'intégration continue et le déploiement continu.
- De **nouvelles approches** émergent, telles que le Test-Driven Development et le Behavior-Driven Development, pour améliorer la qualité du logiciel.

Objectif du GL



- Le génie logiciel visent à produire des **logiciels de haute qualité**
  - qui répondent aux besoins des utilisateurs,
  - tout en respectant les contraintes de temps,
  - de budget et de ressources.

Qualité logiciel



- **Fiabilité** : Assurer que le logiciel fonctionne correctement et de manière prévisible.
- **Efficacité** : Optimiser les performances du logiciel et utiliser les ressources de manière judicieuse.
- **Maintenabilité** : Concevoir le logiciel de manière à faciliter sa mise à jour et son amélioration.
- **Utilisabilité** : Créer des interfaces conviviales et faciles à utiliser pour les utilisateurs finaux.
- **Portabilité** : Assurer que le logiciel peut fonctionner sur différentes plateformes et environnements.
- **Sécurité** : Protéger le logiciel contre les menaces et les vulnérabilités.

Enjeux du GL

- Les enjeux du génie logiciel sont multiples et variés, et ils évoluent constamment avec les avancées technologiques et les besoins changeants de la société.
- Voici quelques-uns des principaux enjeux :
  - Qualité du logiciel ;
  - Complexité croissante ;
  - Évolutivité et maintenance ;
  - Gestion de projet ;
  - Collaboration et communication ;
  - Sécurité ;
  - Conformité réglementaire ;
  - Innovation et compétitivité.

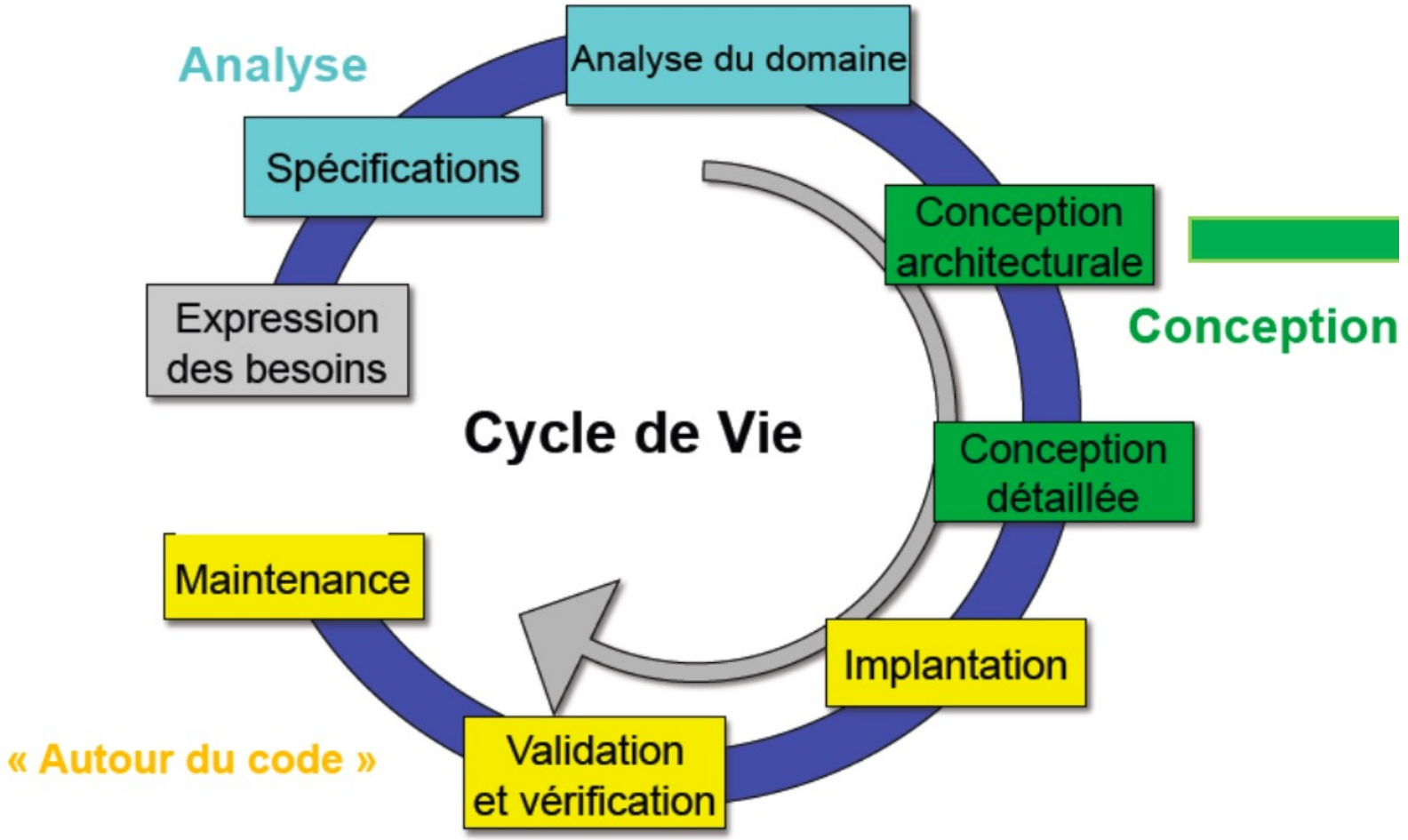


Activités du GL

1. Définition des besoins
2. Spécification
3. Analyse des besoins
4. Conception
5. Programmation
6. Tests
7. Validation et vérification
8. Livraison
9. Supports et formation
10. Maintenance
11. Évolution

- Pour chaque activité : Utilisation et production de documents

Cycle de vie


Analyse
Analyse du domaine
Spécifications
Conception architecturale
Conception
Expression des besoins
Cycle de Vie
Conception détaillée
Maintenance
Implantation
« Autour du code »
Validation et vérification

Modèles de cycle de vie

1. Modèle en Cascade
2. Modèle en V
3. Modèle Itérative et incrémentale
4. Modèle Spirale

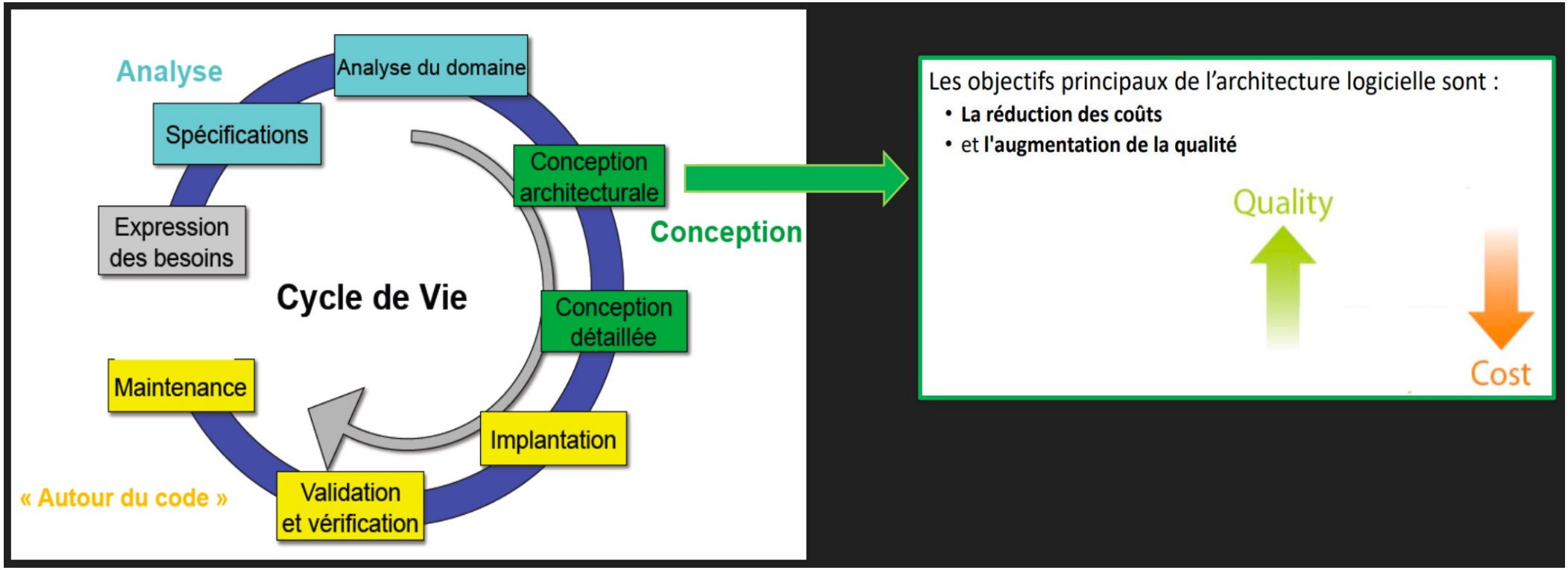
Méthodes du GL

1. Processus UP : 2TUP
2. Méthode Agile
3. Méthode SCRUM
4. Pratiques DevOps
5. CI/CD

Définition 2 de Logiciel

- Un « logiciel » est l'« ensemble des programmes, procédés et règles, et éventuellement de la documentation, relatifs au fonctionnement d'un ensemble de traitement de données ».
- Il s'agit donc
  - d'un ensemble plus ou moins complexe d'instructions,
  - écrites dans un langage de programmation (qui peut être un langage compilé ou un langage interprété)
  - ayant vocation à être exécutées par un processeur, c'est-à-dire par un ordinateur, un terminal mobile (téléphone, tablette, etc.)
  - Pour satisfaire les besoins d'un client

ARCHITECTURE DU LOGICIEL



ARCHITECTURE DU LOGICIEL

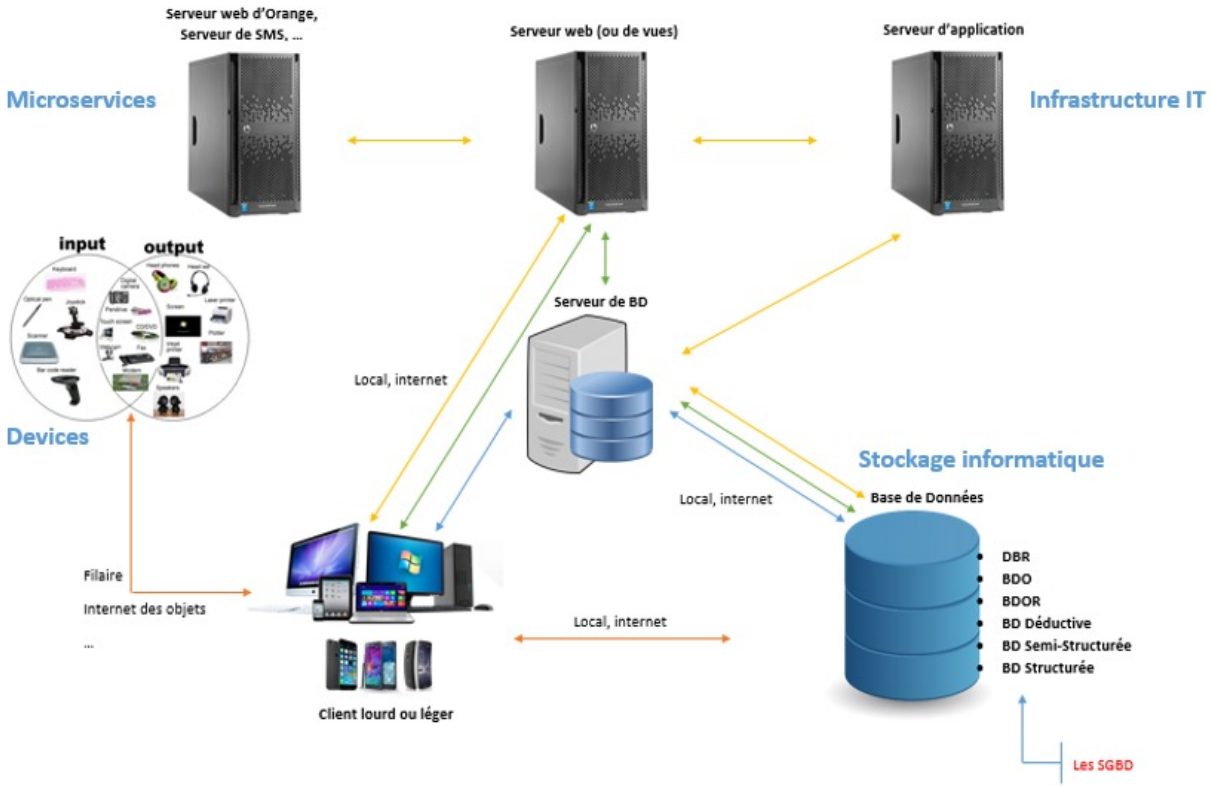


- **L’architecture logicielle** est l'équivalent en ingénierie civile à l’architecture d’un bâtiment.
- C’est la réalisation d’un **produit sous une forme abstraite** avant la production effective.
- La **nature immatérielle** du logiciel (modelé d'informations et non de la matière)
  - rend la frontière entre l'architecture et le produit beaucoup plus floue que dans l'ingénierie civile.

ARCHITECTURE DU LOGICIEL

L'architecture logicielle décrit d'une manière **symbolique** et **schématique**

- les **différents éléments** d'un ou de plusieurs systèmes informatiques ;
- leurs **interrelations** ;
- et leurs **interactions**.



ARCHITECTURE DU LOGICIEL

- Les objectifs principaux de l'architecture logicielle sont :
  - La réduction des coûts
  - et l'augmentation de la qualité



ARCHITECTURE DU LOGICIEL



- **La réduction des coûts**

1. la réutilisation de composants logiciels
   - La réalisation d'économies
2. et la diminution de la dégradation du logiciel
   - La diminution du temps de maintenance ;

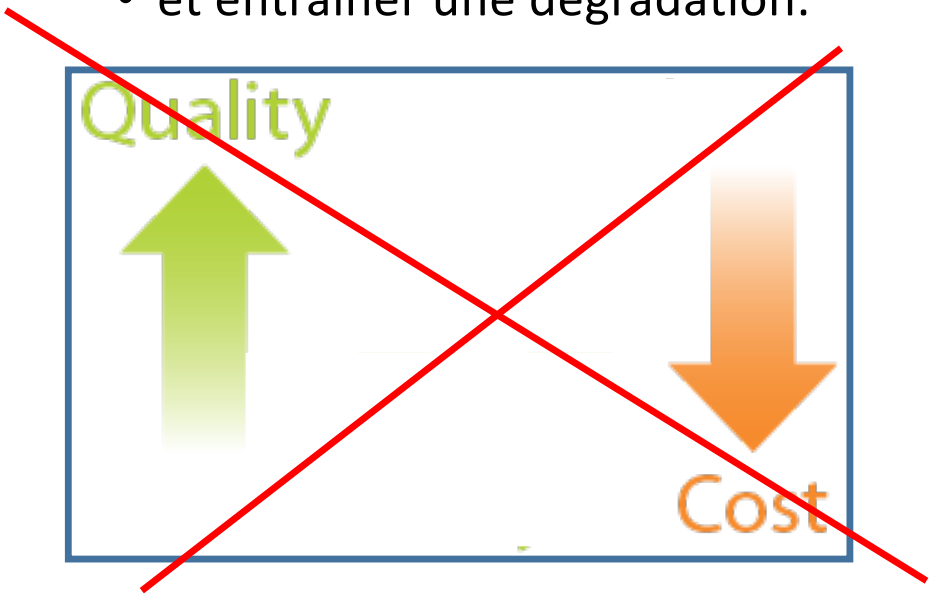
ARCHITECTURE DU LOGICIEL



- **L'augmentation de la qualité** → Les critères de qualité logicielle mesurés avec :
  - ***L'intégrité*** exprime la faculté du logiciel à protéger ses fonctions et ses données d'accès non autorisés.
  - ***La fiabilité*** exprime la faculté du logiciel à gérer correctement ses propres erreurs de fonctionnement en cours d'exécution.
  - ***La maintenabilité*** exprime la simplicité de correction et de modification du logiciel, et même, parfois, la possibilité de modification du logiciel en cours d'exécution.
  - ***La réutilisabilité*** exprime la capacité de concevoir le logiciel avec des composants déjà conçus tout en permettant la réutilisation simple de ses propres composants pour le développement d'autres logiciels.
  - ***L'extensibilité*** exprime la possibilité d'étendre simplement les fonctionnalités d'un logiciel sans compromettre son intégrité et sa fiabilité.
  - ***…***

ARCHITECTURE DU LOGICIEL

- Une architecture **faible ou absente**
  - entraîner de graves problèmes lors de la maintenance du logiciel.
- La modification d'un **logiciel mal architecturé**
  - déstabiliser la structure de celui-ci
  - et entraîner une dégradation.



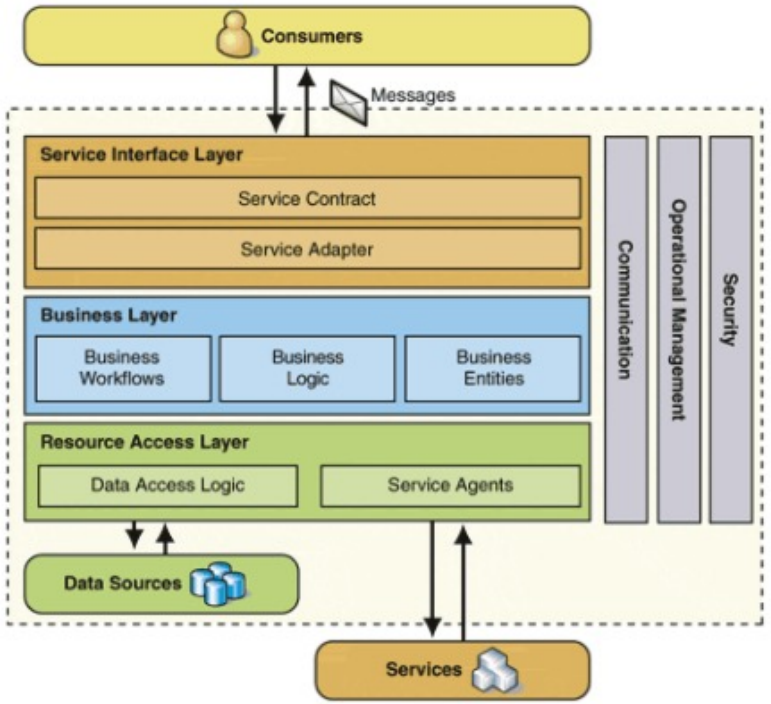


Architectures Monolithiques

- L'origine de la notion d'architecture logicielle remonte à la fin des années 1960 avec l'invention de la programmation structurée.
- Depuis cette date plusieurs **styles d'architecture logicielle** ont vu le jour :
  - Architecture en couches ;
  - Architecture centrée sur les données ;
  - Architecture en flot de données ;
  - Architecture orientée objets ;
  - Architecture orientée agents ;
  - Client-serveur ;
  - Modèle N-tiers ;
  - Model-View-Controller (MVC) ;
  - Peer-to-peer ;
  - …

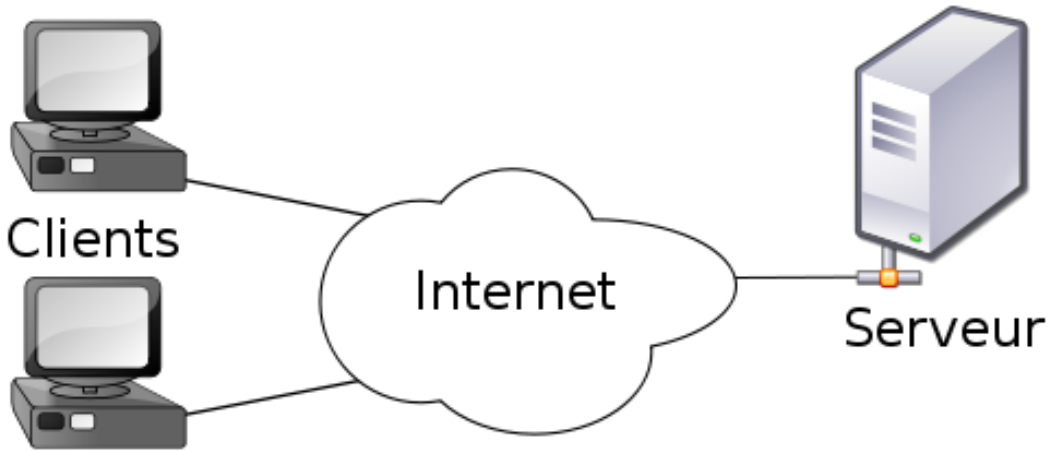
Architectures Monolithiques

- L'origine de la notion d'architecture logicielle remonte à la fin des années 1960 avec l'invention de la programmation structurée.
- Depuis cette date plusieurs **styles d'architecture logicielle** ont vu le jour :
  - **Architecture en couches ;**
  - Architecture centrée sur les données ;
  - Architecture en flot de données ;
  - Architecture orientée objets ;
  - Architecture orientée agents ;
  - Client-serveur ;
  - Modèle N-tiers ;
  - Model-View-Controller (MVC) ;
  - Peer-to-peer ;
  - …



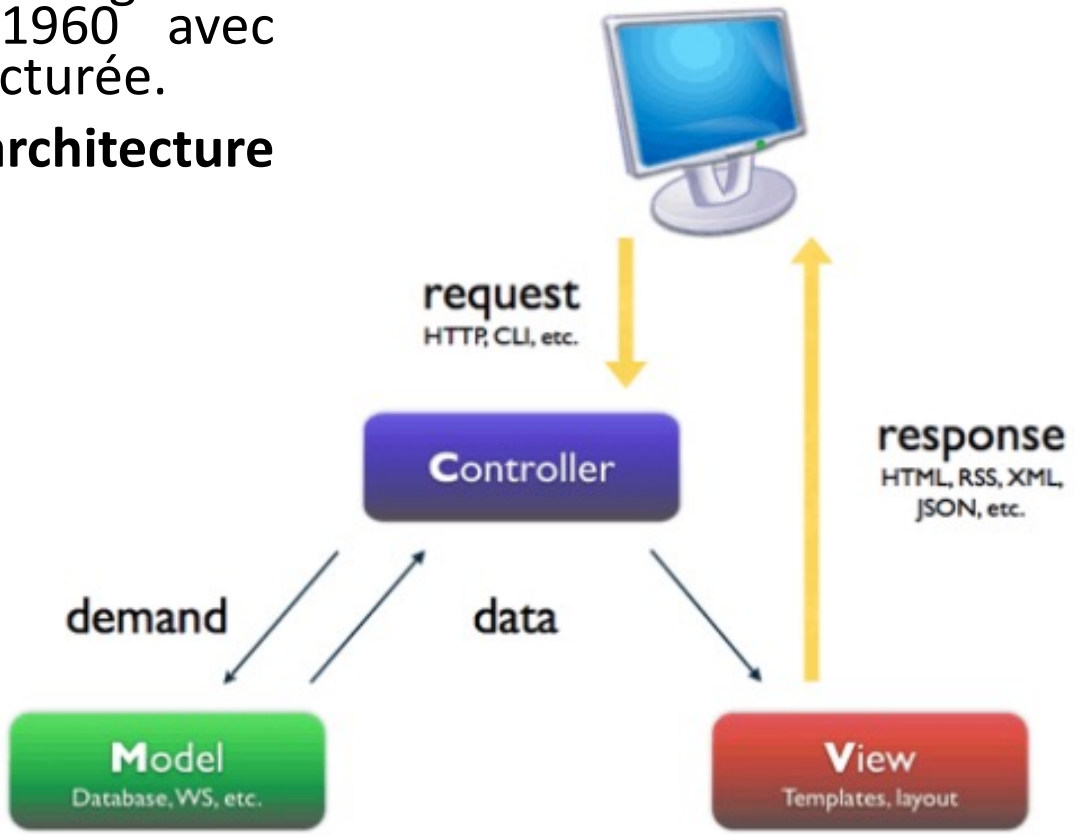
Architectures Monolithiques

- L'origine de la notion d'architecture logicielle remonte à la fin des années 1960 avec l'invention de la programmation structurée.
- Depuis cette date plusieurs **styles d'architecture logicielle** ont vu le jour :
  - Architecture en couches ;
  - Architecture centrée sur les données ;
  - Architecture en flot de données ;
  - Architecture orientée objets ;
  - Architecture orientée agents ;
  - **Client-serveur ;**
  - Modèle N-tiers ;
  - Model-View-Controller (MVC) ;
  - Peer-to-peer ;
  - …

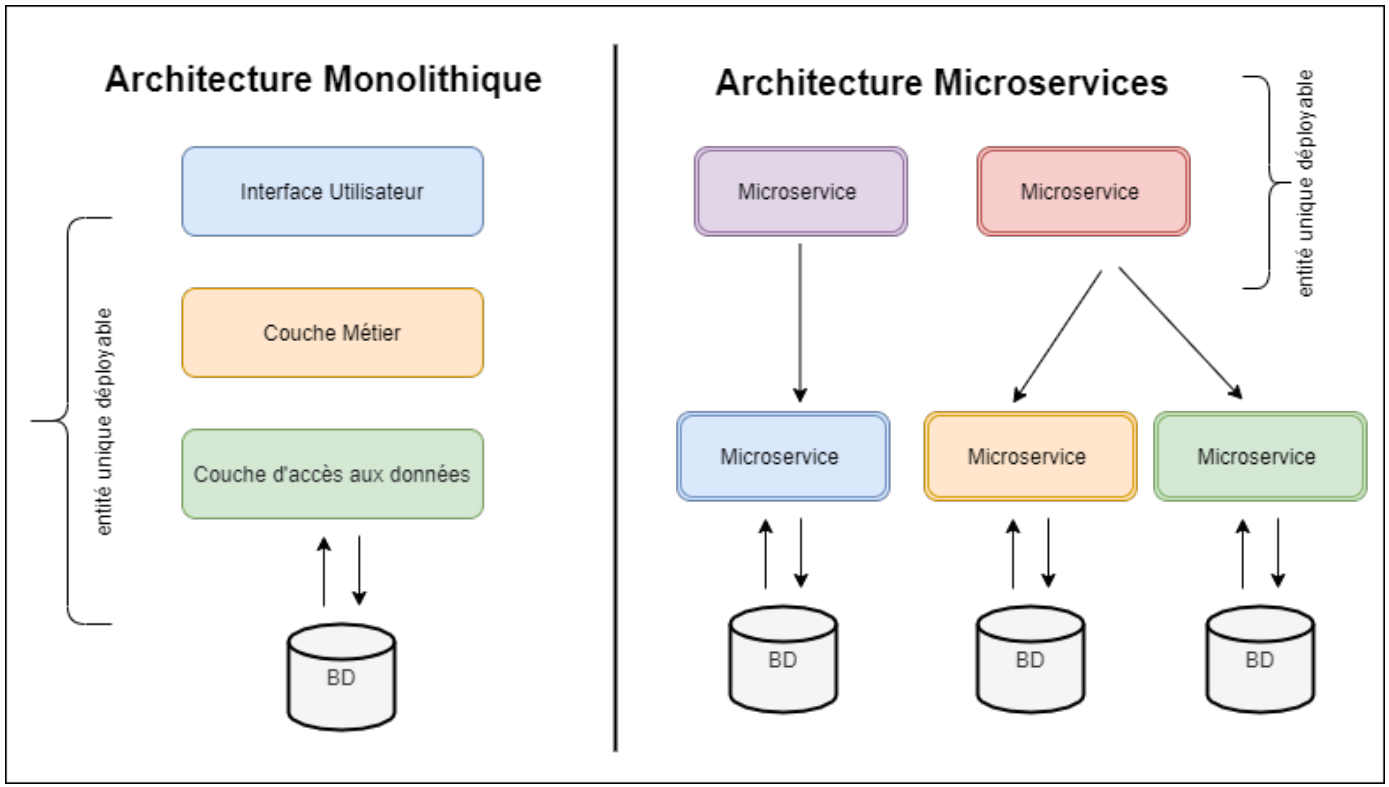


Architectures Monolithiques

- L'origine de la notion d'architecture logicielle remonte à la fin des années 1960 avec l'invention de la programmation structurée.
- Depuis cette date plusieurs **styles d'architecture logicielle** ont vu le jour :
  - Architecture en couches ;
  - Architecture centrée sur les données ;
  - Architecture en flot de données ;
  - Architecture orientée objets ;
  - Architecture orientée agents ;
  - Client-serveur ;
  - Modèle N-tiers ;
  - **Model-View-Controller (MVC) ;**
  - Peer-to-peer ;
  - …



Architecture Microservices



Objectif Général

- Vous permet de connaitre Génie Logiciel, ses méthodes, ses bonnes pratiques et outils. Ce cours met une attention particulière sur l'architecture du logiciel, qui est un des outils important du GL.
- **THÈME :** LA FABRICATION D'UN LOGICIEL POUR LA GESTION DES RECRUTEMENTS DU PERSONNEL DE L'UASZ

Objectifs spécifiques

- **À l'issu de ce cours, vous serez capable:**
  - De définir le logiciel, le génie logiciel et de décrire leur histoire ;
  - De lister et de décrire les activités du génie logiciel ;
  - D'utiliser et produire des documents pour chaque activité du GL ;
  - De décrire et de comparer les différentes modèles de cycles de vie du logiciel ;
  - De décrire les méthodes et pratiques du génie logiciel ;
  - De définir l'Architecture Logicielle et de préciser sa place dans le Génie Logiciel ;
  - De comparer les différents styles d'architectures logicielles.

Pré requis

- Pour suivre ce cours vous devez avoir:
  1. Algorithme et programmation
  2. Analyse et Conception de systèmes d'information
  3. Les diagrammes UML

Stratégie pédagogique

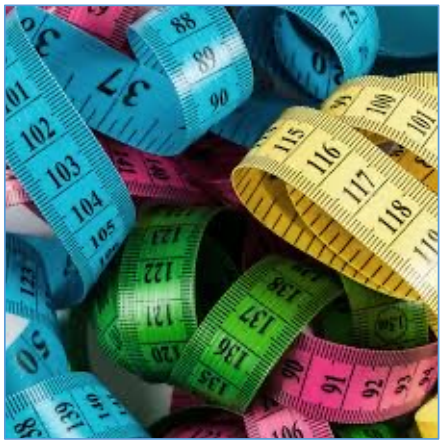
- Alternance de **cours théoriques** sous forme d'exposé et des **Travaux pratiques**.
- **Ateliers de génie logiciel** pour : « LA FABRICATION D'UN LOGICIEL POUR LA GESTION DES RECRUTEMENTS DU PERSONNEL DE L'UASZ ».

PLAN

1. INTRODUCTION
2. ACTIVITÉS DU GL
3. MODÈLES DU GL
4. MÉTHODES DU GL
5. ARCHITECTURES LOGICIELLES
6. CONCLUSION

PLAN

1. INTRODUCTION
2. **ACTIVITÉS DU GL**
3. MODÈLES DU GL
4. MÉTHODES DU GL
5. ARCHITECTURES LOGICIELLES
6. CONCLUSION



...



Les activités du GL

- Ensemble d'activités successives, organisées en vue de la production d'un logiciel En pratique :
  - Pas de processus idéal
  - Choix du processus en fonction des contraintes (taille des équipes, temps, qualité...)
  - Adaptation de « processus types » aux besoins réels

Les activités du GL

1. Définition des besoins
2. Spécification et Analyse des besoins
3. Conception
4. Programmation
5. Tests : Validation et vérification
6. Livraison
7. Supports et formation
8. Maintenance
9. Évolution
10. Sécurité, Gestion de projet , Assurance qualité, …

- Pour chaque activité : Utilisation et production de documents

Définition des besoins

- Recueillir les exigences des utilisateurs.

-> Le **cahier des charges** écrire dans le langage du client

Cahier de charge

- Un cahier des charges est un document essentiel dans le cadre de la gestion de projet génie logiciel. Il détaille les exigences et les attentes pour un projet ou un produit.

Les activités du GL – Spécification et Analyse des besoins

- Traduction des besoins dans un langage *plus informatique -> Ce que doit faire le logiciel (et non comment il le fait)*

Diagrammes UML et Document

- Diagramme de cas d’utilisation
- Diagramme de états-transitions
- Diagramme d’activités
- Diagramme de séquences

-> **Le document de spécification et d’analyse**

Les activités du GL - Conception

- Développer l'architecture du logiciel.
- Créer des diagrammes de flux, des modèles de données et des maquettes d'interface utilisateur.
- Définir les modules, les composants et leurs interactions.

Diagrammes UML et Document

- Conception générale
  - Diagramme de package
  - Diagramme de composants
  - Diagramme de déploiement
- Conception détaillée
  - Diagramme de classes
  - Diagramme d'objets

  -> **Le document de Conception**

Les activités du GL - Programmation

- Écrire le code selon les spécifications de conception.
- Utiliser des langages de programmation appropriés
- Suivre les meilleures pratiques de codage.
- Intégrer les composants logiciels.

-> Traduction de la conception en code

Les activités du GL - Validation et Vérification

- Avons-nous construit le bon logiciel ? ->Validé
- Avons-nous bien construit le logiciel ? ->Vérifié

Tests

- Élaborer des plans de test et des cas de test.
- Effectuer des tests unitaires, d'intégration, de système et de régression.
- Identifier et corriger les bugs et les défauts.

Les activités du GL - Livraison

- Préparer et exécuter les plans de déploiement.
- Configurer les environnements de production.
- Assurer la transition et la formation des utilisateurs finaux.

Les activités du GL - Support, formation

- Créer et maintenir la documentation technique et utilisateur.
- Documenter le code, les architectures, les interfaces et les processus.
- Rédiger des manuels d'utilisation et des guides de déploiement.

Les activités du GL - Maintenance

- Corriger les erreurs et les bugs détectés après la mise en production.
- Améliorer les fonctionnalités existantes en fonction des nouvelles exigences.
- Assurer la compatibilité avec les nouvelles technologies et environnements.

Les activités du GL - Evolution

- Utiliser des systèmes de contrôle de version pour gérer le code source.
- Suivre les versions du logiciel et les configurations associées.
- Gérer les branches, les fusions et les versions de release.

Les activités du GL - Sécurité

- Intégrer des pratiques de sécurité dans le cycle de développement.
- Effectuer des analyses de vulnérabilité et des tests de pénétration.
- Mettre en œuvre des contrôles de sécurité et des mécanismes de protection.

Les activités du GL - Gestion de projet

- Planifier et suivre les activités du projet.
- Gérer les ressources, les budgets et les calendriers.
- Assurer la communication entre les parties prenantes.

Les activités du GL - Assurance qualité

- Mettre en place des processus pour garantir la qualité du logiciel.
- Effectuer des revues de code, des audits et des inspections.
- Utiliser des outils d'analyse statique et dynamique pour améliorer la qualité.

Cycle de vie du logiciel

- Le cycle de vie désigne toutes les **étapes** du développement d'un logiciel, de sa conception à sa disparition.
- Les modèles décrivent les liens, les relations entre les différentes **étapes** du cycle de vie du logiciel.
- Les méthodes permettent de mettre en œuvre un développement logiciel selon un modèle en organisant les différentes **étapes** du cycle de vie du logiciel.

PLAN

1. INTRODUCTION
2. ACTIVITÉS DU GL
3. **MODÈLES DU GL**
4. MÉTHODES DU GL
5. ARCHITECTURES LOGICIELLES
6. CONCLUSION

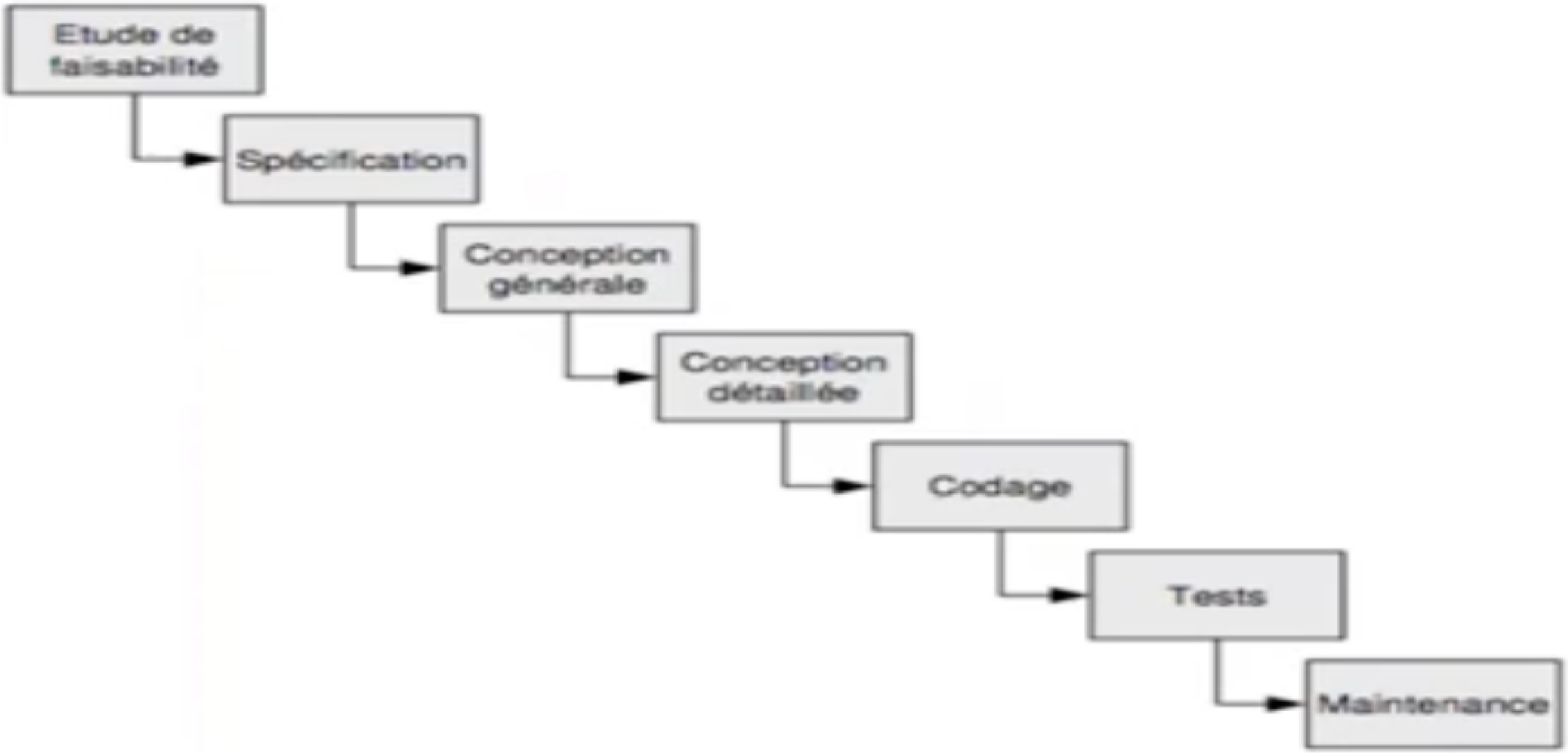
Modèles de cycle de vie du logiciel

1. **Linéaire**
    1. Modèle en Cascade
    2. Modèle en V
2. **Non linéaire**
    1. Modèle Itérative et incrémentale
    2. Modèle Spirale

Modèle en Cascade

- Le **modèle en cascade** (ou de **cycle de vie en cascade)** est l'une des premières modèles de développement logiciel formalisées.
- Il s'agit d'un modèle séquentiel où le processus de développement est **divisé en phases distinctes**.
- Chaque phase doit être complétée avant que la suivante ne commence, sans retour en arrière possible.
- Ce modèle est linéaire et rigide, et il est souvent comparé à une chute d'eau, d'où son nom : modèle en cascade.

Phases du Modèle en Cascade



Avantages du Modèle en Cascade

- **Simplicité et clarté** : La séquence linéaire des étapes est facile à comprendre et à gérer.
- **Documentation complète** : Chaque phase produit une documentation détaillée, ce qui facilite la traçabilité et la gestion des exigences.
- **Facilité de gestion** : Les étapes clairement définies facilitent la planification et le suivi des progrès.

Inconvénients du Modèle en Cascade

- **Rigidité** : Le modèle ne permet pas de revenir en arrière pour modifier ou améliorer les phases précédentes une fois qu'elles sont terminées.
- **Adaptabilité limitée** : Difficile à adapter aux changements des exigences en cours de développement.
- **Risque de décalage des besoins** : Les besoins des utilisateurs peuvent évoluer pendant le développement, mais le modèle en cascade ne gère pas ces changements.
- **Livraison tardive** : Le produit final n'est livré qu'à la fin du cycle de développement, ce qui peut retarder la détection de problèmes majeurs jusqu'à la phase de test.

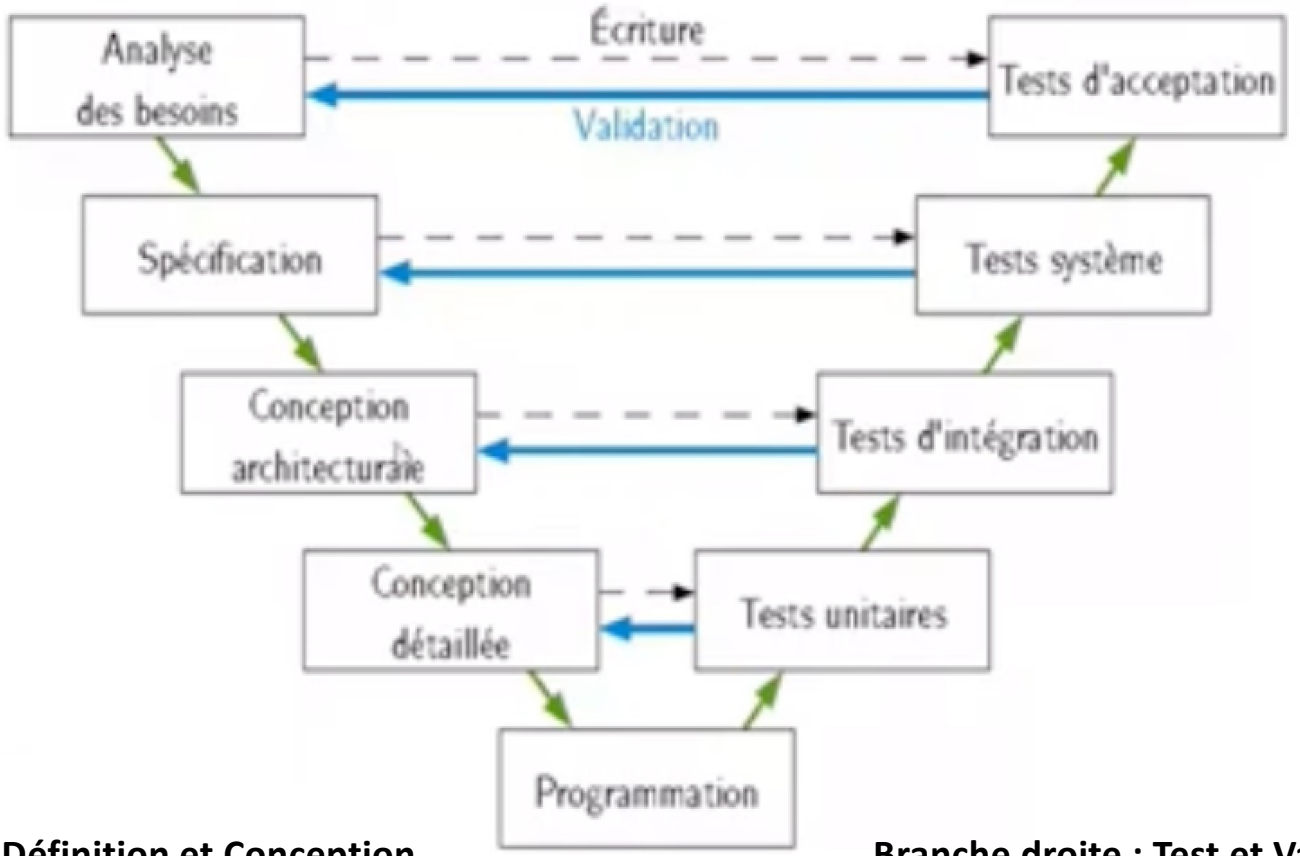
Application du Modèle en Cascade

- Le modèle en cascade est souvent utilisé dans les projets où les exigences sont bien comprises dès le début et sont peu susceptibles de changer ;
- Cependant, dans les environnements où les exigences peuvent évoluer rapidement et où une rétroaction continue est essentielle, d'autres méthodes sont préférées.

Modèle en V

- Le **modèle en V** est une extension du modèle en cascade, visant à adresser certaines de ses limitations, notamment en termes de test et de validation.
- Il est appelé modèle en V en raison de sa forme qui ressemble à la lettre "V", avec la branche gauche représentant les phases de définition et de conception, et la branche droite les phases de test et de validation correspondantes.
- Le point central du "V" représente l'implémentation du système.

Phases du Modèle en V



Branche gauche : Définition et Conception

Branche droite : Test et Validation

Avantages du Modèle en V

- **Clarté des phases de test** : Chaque phase de définition et de conception a une phase de test correspondante, ce qui permet de vérifier systématiquement chaque aspect du système.
- **Documentation détaillée** : Comme le modèle en cascade, il produit une documentation complète à chaque étape.
- **Détection précoce des défauts** : Les tests planifiés dès le début permettent de détecter les défauts plus tôt dans le cycle de développement.

Inconvénients du Modèle en V

- **Rigidité** : Comme le modèle en cascade, il est linéaire et rigide, ne permettant pas facilement de revenir en arrière pour modifier les phases précédentes.
- **Adaptabilité limitée** : Il gère mal les changements d'exigences en cours de développement.
- **Livraison tardive** : Le produit final n'est livré qu'après la validation et la vérification complètes, ce qui peut retarder la détection de problèmes majeurs.

Application du Modèle en V

- Le modèle en V est souvent utilisé dans les projets où la qualité et la rigueur des tests sont critiques, comme dans le développement de systèmes embarqués, de logiciels médicaux, ou de systèmes aéronautiques et militaires.
- Il est particulièrement adapté lorsque les exigences sont bien définies dès le début et ne sont pas susceptibles de changer de manière significative au cours du développement.

Modèle Itérative et incrémentale

- Le modèle **itératif et incrémental** est une approche de développement logiciel qui combine les éléments des modèles itératifs et des modèles incrémentaux.
- Il vise à répondre aux limitations des modèles traditionnels comme le modèle en cascade, en permettant une plus grande flexibilité et adaptabilité aux changements d'exigences au cours du développement.
- Cette méthode se caractérise par des cycles de développement courts et répétitifs (itérations), avec des versions du produit de plus en plus complètes (incréments) à chaque cycle.

Caractéristiques du Modèle Itératif et Incrémental

1. **Itérations** :
    1. Chaque cycle de développement (ou itération) produit une version partielle mais fonctionnelle du produit.
    2. Les itérations sont courtes, généralement de quelques semaines à quelques mois.
2. **Incréments** :
    1. À chaque itération, des fonctionnalités ou des améliorations sont ajoutées au produit.
    2. Chaque incrément est une version plus complète et plus fonctionnelle du logiciel.

Phases du Modèle Itérative et incrémentale



Avantages du Modèle Itératif et Incrémental

- **Flexibilité** : Les changements peuvent être intégrés facilement à chaque itération.
- **Feedback continu** : Les utilisateurs et les parties prenantes peuvent donner leur avis après chaque itération, permettant des ajustements rapides.
- **Réduction des risques** : Les problèmes peuvent être identifiés et résolus plus tôt grâce aux cycles de développement courts.
- **Livraison progressive** : Les versions fonctionnelles du produit sont disponibles à la fin de chaque itération, offrant des opportunités de mise en production anticipée.

Inconvénients du Modèle Itératif et Incrémental

- **Complexité de gestion** : Nécessite une planification et une gestion rigoureuses pour chaque itération.
- **Risque de dérive des exigences** : Les ajustements fréquents peuvent entraîner des changements constants des exigences.
- **Coût potentiel** : La révision et l'ajustement continus peuvent engendrer des coûts supplémentaires en termes de temps et de ressources.

Application du Modèle Itératif et Incrémental

- Le modèle itératif et incrémental est largement utilisé dans les environnements où les exigences sont susceptibles de changer et où il est essentiel d'obtenir des retours rapides des utilisateurs.
- Il est particulièrement adapté aux projets de développement de logiciels complexes, aux projets de développement agile et aux environnements où la rapidité de mise sur le marché est cruciale.

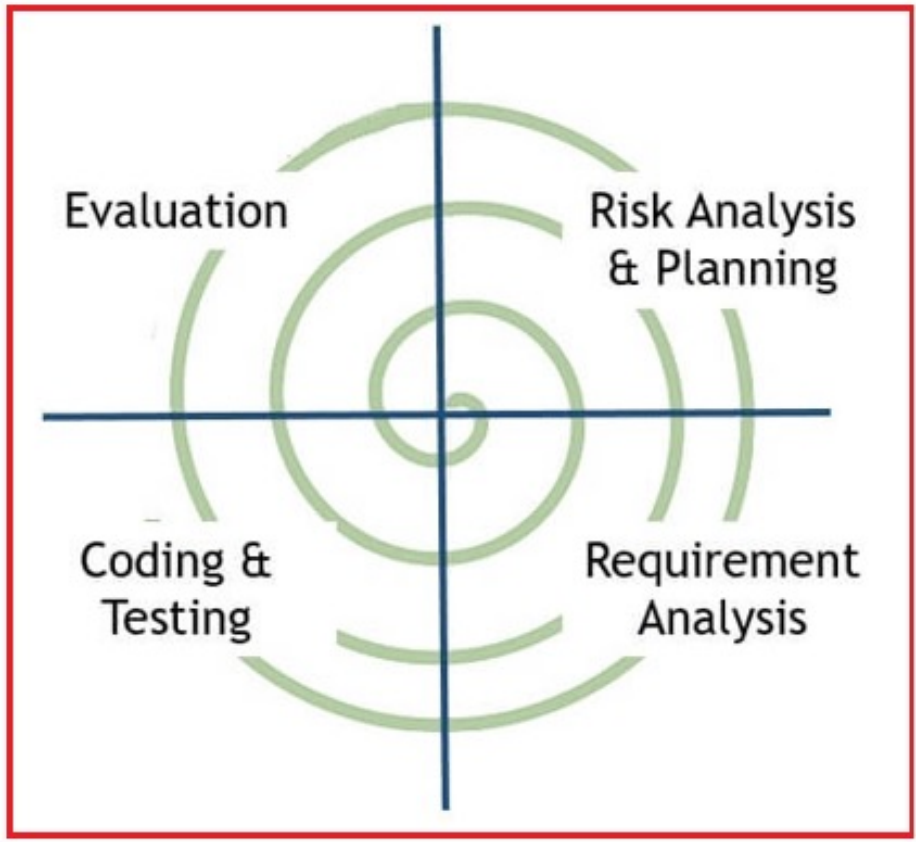
Modèle Spirale

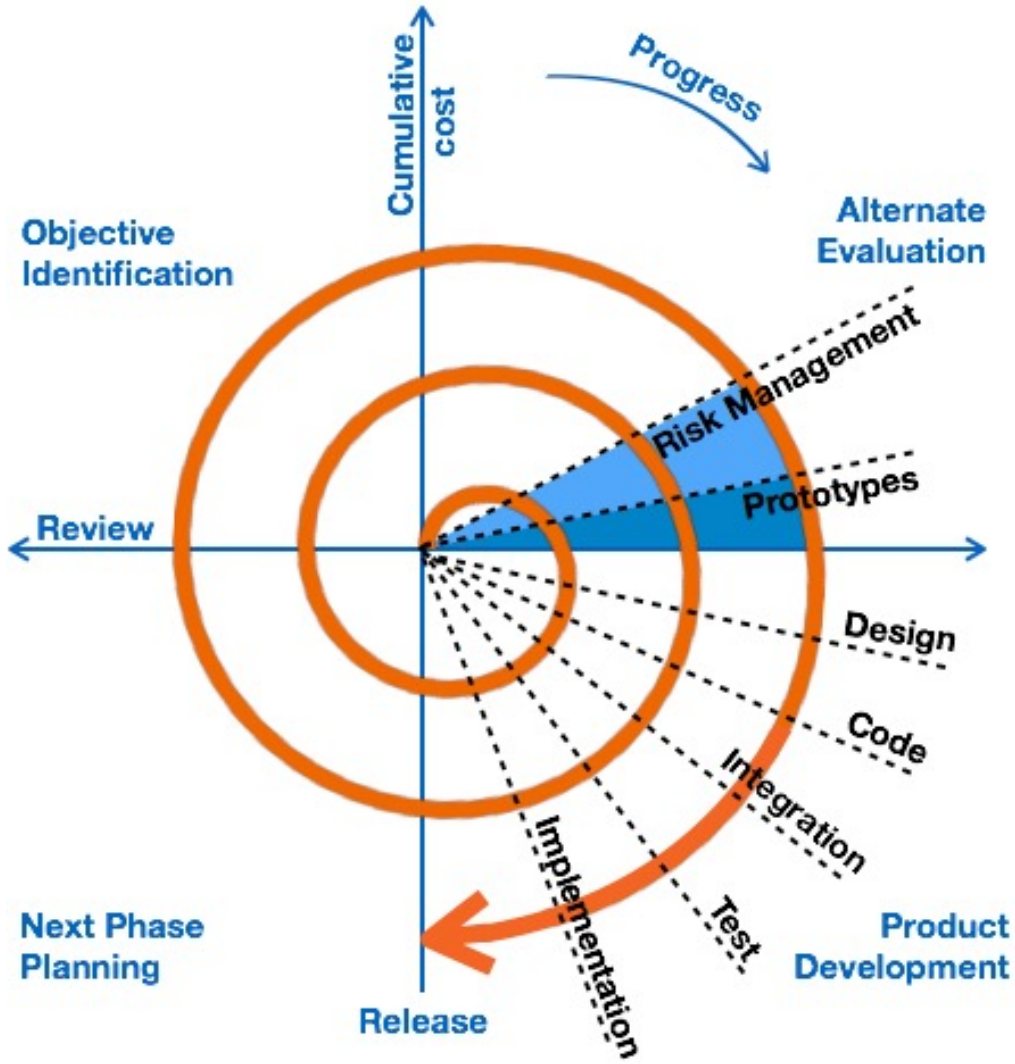
- Le **modèle en spirale** combine des aspects de la conception itérative et du modèle en cascade, tout en introduisant une forte composante d'évaluation des risques.
- Ce modèle met l'accent sur la gestion des risques à chaque phase du développement. Le processus se déroule en spirales successives, chaque boucle de la spirale représentant une phase du projet.

Caractéristiques du Modèle en Spirale

1. **Itératif et incrémental** : Comme le modèle itératif, le modèle en spirale permet des cycles répétés de développement. Chaque itération passe par les phases de planification, de conception, de développement et de validation, produisant des incréments progressifs du produit.
2. **Gestion des risques** : La gestion des risques est au cœur du modèle en spirale. À chaque cycle, les risques sont identifiés, analysés et des mesures sont prises pour les atténuer.
3. **Flexibilité** : Le modèle est flexible et permet d'ajuster les exigences et les solutions au fur et à mesure que le projet progresse, en fonction des retours d'expérience et des risques identifiés.

Phases du Modèle Spiral





Avantages du Modèle en Spirale

- **Gestion proactive des risques** : La gestion systématique des risques à chaque itération permet d'identifier et de résoudre les problèmes potentiels dès les premières étapes du développement.
- **Flexibilité et adaptabilité** : Les exigences et les solutions peuvent être ajustées au fur et à mesure que le projet progresse, permettant une meilleure adaptation aux changements.
- **Feedback continu** : Les utilisateurs et les parties prenantes peuvent fournir des retours d'expérience à chaque itération, améliorant ainsi la qualité du produit final.
- **Documentation et suivi rigoureux** : Chaque itération produit une documentation détaillée, facilitant la traçabilité et la gestion du projet.

Inconvénients du Modèle en Spirale

- **Complexité et coût** : La gestion des risques et la planification détaillée à chaque itération peuvent rendre le modèle complexe et coûteux à mettre en œuvre.
- **Nécessité d'une expertise en gestion des risques** : Le succès du modèle dépend fortement de la capacité de l'équipe à identifier et à gérer les risques de manière efficace.
- **Temps et ressources** : Les cycles répétés de planification, de développement et de vérification peuvent nécessiter plus de temps et de ressources par rapport à d'autres modèles.

Application du Modèle en Spirale

- Le modèle en spirale est adapté aux **projets de grande envergure et complexes**, où les **risques sont élevés** et les **exigences sont susceptibles de changer** au cours du développement.
- Il est souvent utilisé dans les secteurs où la gestion des risques est cruciale, comme les systèmes embarqués, les logiciels de sécurité critique, et les projets de défense et aérospatiaux.

Modèle UP

- En 1992, Rational Software publie le modèle Booch de développement orientée objets.
- En 1997, le « langage de modélisation unifié », UML, devient un standard ;
- En 1998, à la suite de l'acquisition de plusieurs autres sociétés spécialisées dans les outils de génie logiciel, Rational Software améliore sa méthode et la renomme « *Rational Unified Process* (RUP)»
- En 1999, « Le processus unifié de développement logiciel » est publié dans le but de diffuser au grand public les idées à la base de RUP.

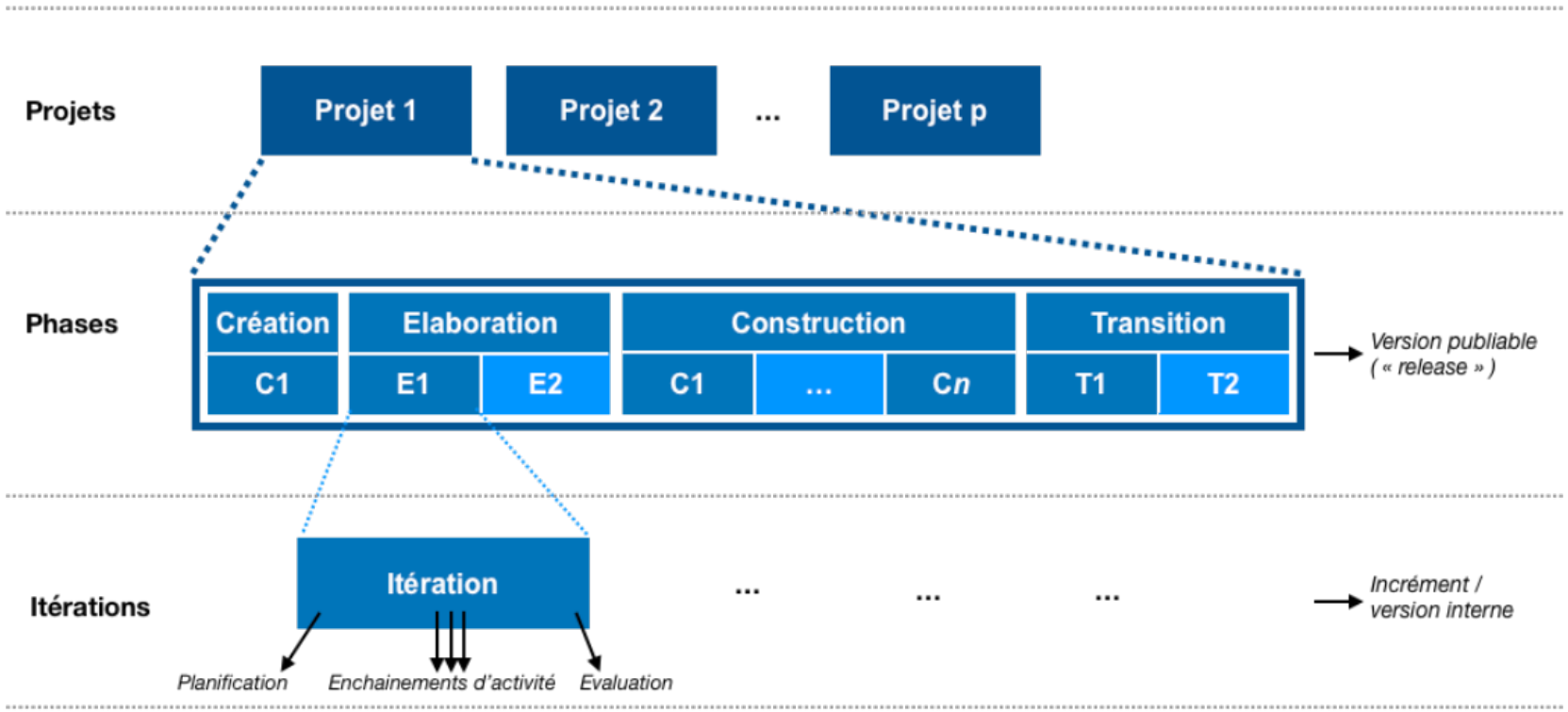
Caractéristiques de UP

- **Itératif et Incrémental :** Le développement est divisé en itérations courtes et gérables. Chaque itération produit un incrément du logiciel, ajoutant des fonctionnalités progressivement.
- **Centré sur l'Architecture :** L'architecture du système est définie dès les premières phases et sert de cadre pour les développements ultérieurs.
- **Conduite par les Cas d'Utilisation :** Les cas d'utilisation (use cases) sont utilisés pour capturer les exigences fonctionnelles et guider le développement.
- **Basé sur des Modèles :** Utilisation extensive de la modélisation, notamment avec le langage UML (Unified Modeling Language).

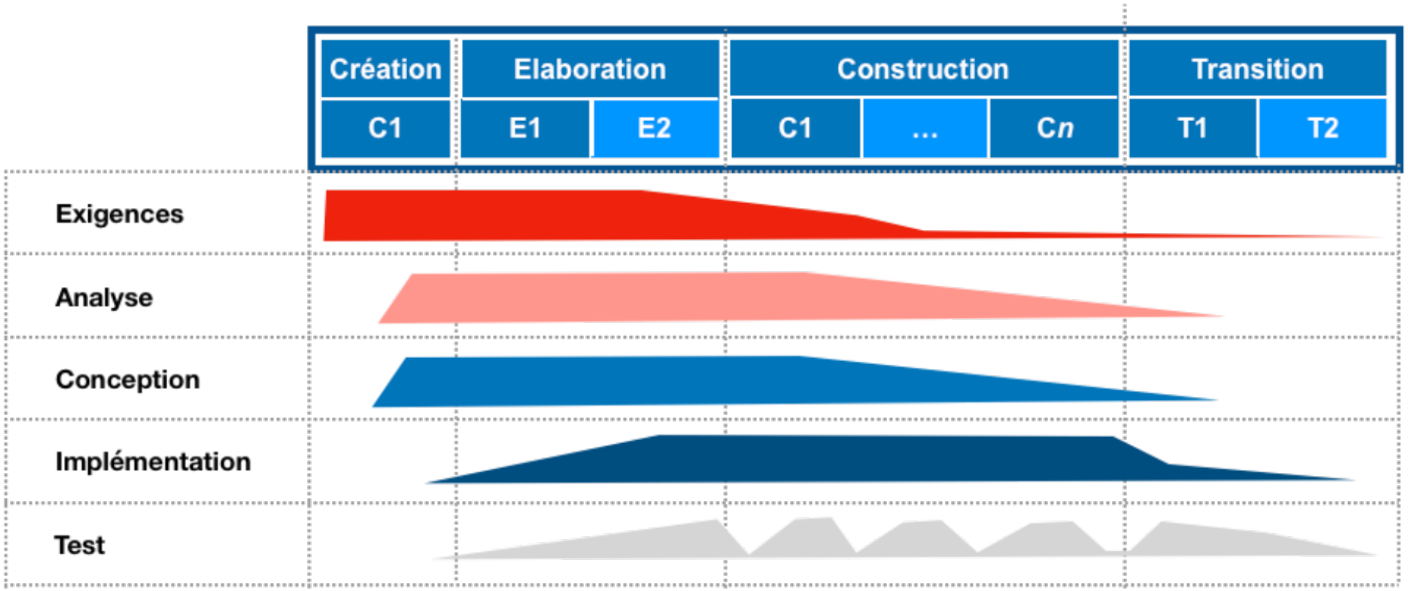
Phases du Processus UP

- **Création :**
  - **Objectif** : Définir le périmètre du projet, identifier les risques majeurs et obtenir l'approbation initiale.
  - **Livrables** : Vision du projet, cas d'utilisation principaux, plan de projet initial, prototype éventuellement.
- **Élaboration :**
  - **Objectif** : Affiner les exigences, développer l'architecture de base et réduire les risques les plus critiques.
  - **Livrables** : Modèle d'architecture, documentation des cas d'utilisation détaillés, plan de projet mis à jour.
- **Construction :**
  - **Objectif** : Développer le système complet par itérations successives, intégrant toutes les fonctionnalités prévues.
  - **Livrables** : Code source, documentation technique, tests unitaires et d'intégration.
- **Transition :**
  - **Objectif** : Déployer le système chez les utilisateurs finaux et corriger les défauts restants.
  - **Livrables** : Version finale du produit, documentation utilisateur, support et maintenance initiale.

Phases du Processus UP



Enchainements d'activités



Avantages

- **Adaptabilité :** Permet de répondre aux changements de besoins des utilisateurs grâce à son approche itérative.
- **Réduction des Risques :** Identifie et traite les risques dès le début du projet.
- **Qualité :** Favorise la qualité à travers des pratiques de modélisation et de test rigoureuses.
- **Visibilité :** Offre une vision claire de l'avancement du projet grâce à des itérations régulières.

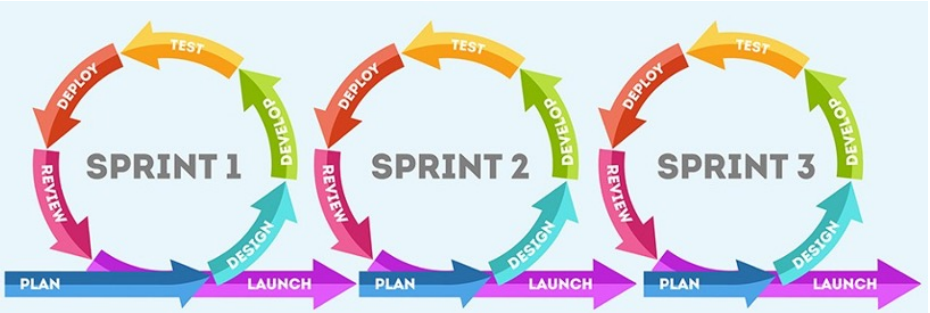
Inconvénients

- **Complexité :** Peut être complexe à mettre en œuvre, nécessitant une formation et une compréhension approfondie de ses principes.
- **Coût :** Peut engendrer des coûts plus élevés en raison de la rigueur des processus et des outils utilisés.

Les méthodes UP

XUP	*eXtreme Unified Process*	hybride intégrant UP avec Extreme programming.
AUP	*Agile Unified Process*	hybride combinant un sous-ensemble de UP avec des préceptes des méthodes agile
2TUP	*two Tracks Unified Process*	susceptible de prendre en compte les aléas et contraintes liées aux changements perpétuels et rapides des SI des entreprises
EssUP	*Essential Unified Process*	alternative allégée de UP intégrant des préceptes agiles. EssUP est composée d'un ensemble de pratiques, dites "essentielles", qui peuvent être librement sélectionnées et combinées offrant ainsi une grande flexibilité dans son application

Modèle Agile



- Le modèle agile vise à améliorer la flexibilité, la collaboration et la capacité à répondre aux changements dans le développement logiciel.
- il met l'accent sur la livraison de petites portions de fonctionnalité et sur la communication continue avec les parties prenantes.
- Le document fondateur de modèle agile se base sur :
  - **quatre** valeurs fondamentales ;
  - et **douze** principes.

4 Valeurs du modèle Agile

1. Les individus et leurs interactions plus que les processus et les outils ;
2. Des logiciels opérationnels plus qu'une documentation exhaustive ;
3. La collaboration avec les clients plus que la négociation contractuelle ;
4. L'adaptation au changement plus que le suivi d'un plan.

12 Principes du modèle Agile

1. **Satisfaction du client par la livraison précoce et continue de logiciels utiles.**
   - Cela signifie que l'objectif principal est de fournir rapidement des fonctionnalités qui apportent de la valeur au client, plutôt que de se concentrer sur des documents ou des processus.
2. **Accueil des changements de besoins, même tard dans le projet.**
   - Les processus agiles visent à être flexibles face aux changements, et considèrent que les exigences évoluent souvent pendant le développement.
3. **Livraison fréquente du logiciel fonctionnel, toutes les quelques semaines à quelques mois, avec une préférence pour les périodes les plus courtes.**
   - Cette pratique permet de s'assurer que les équipes restent alignées sur les besoins du client et peuvent ajuster le produit en fonction des retours obtenus.

12 Principes du modèle Agile

4. **Collaboration quotidienne entre les personnes impliquées tout au long du projet.**
   - L'interaction régulière entre les membres de l'équipe et avec les parties prenantes est cruciale pour le succès du projet.
5. **Construction de projets autour de personnes motivées.**
   - Les individus motivés sont la clé pour atteindre les objectifs du projet. Il est donc important de créer un environnement où ils peuvent exceller.
6. **Face-à-face est la méthode de communication la plus efficace pour une équipe agile.**
   - La communication directe aide à réduire les malentendus et à faciliter la résolution rapide des problèmes.

12 Principes du modèle Agile

7. **Le logiciel fonctionnel est la principale mesure de progrès.**
   - Au lieu de se concentrer sur la documentation exhaustive ou les étapes de processus, les équipes agiles mesurent leur succès par la capacité à livrer un logiciel fonctionnel.
8. **Maintien d'un rythme de développement soutenable.**
   - Les équipes doivent être capables de maintenir un rythme constant de développement indéfiniment.
9. **Attention continue à l'excellence technique et à une bonne conception pour améliorer l'agilité.**
   - Un bon code et une architecture solide facilitent les changements futurs et permettent à l'équipe de répondre rapidement aux besoins changeants.

12 Principes du modèle Agile

10. **Simplicité, c'est-à-dire l'art de maximiser la quantité de travail non effectué, est essentielle.**
    - Éviter la sur-ingénierie et se concentrer sur l'essentiel permet de minimiser le gaspillage et de maximiser la valeur ajoutée.
11. **Les meilleures architectures, exigences et conceptions émergent d'équipes auto-organisées.**
    - Les équipes doivent avoir la liberté de prendre des décisions qui affectent leur travail, tout en restant alignées sur les objectifs du projet.
12. **À intervalles réguliers, l'équipe réfléchit aux moyens de devenir plus efficace, puis ajuste et affine son comportement en conséquence.**
    - L'auto-évaluation et l'amélioration continue sont essentielles pour s'assurer que l'équipe maintient sa performance et son efficacité au fil du temps.

Avantages

- **Flexibilité et Adaptabilité :** Capacité à répondre rapidement aux changements de besoins.
- **Livraison Fréquente :** Livraison régulière de versions fonctionnelles du produit.
- **Amélioration Continue :** Réflexion régulière sur le processus et ajustement pour l'améliorer.
- **Engagement des Parties Prenantes :** Collaboration continue avec les utilisateurs et les parties prenantes.

Inconvénients

- **Nécessité d'un Engagement Constant :** Exige une implication continue des utilisateurs et des parties prenantes.
- **Difficulté à Planifier à Long Terme :** La nature itérative peut rendre la planification à long terme plus difficile.
- **Risque de Manque de Documentation :** Moins de documentation formelle peut poser des problèmes de maintenance à long terme.

Les méthodes Agiles

Scrum	Scrum est l'une des méthodes agiles les plus populaires. Il divise le développement en cycles de travail appelés "Sprints", généralement de 2 à 4 semaines.
XP (Extreme Programming)	hybride combinant un sous-ensemble de UP avec des préceptes des méthodes agile
Kanban	Kanban est une méthode visuelle pour la gestion de projet qui se concentre sur l'optimisation de la flux de travail.
Lean Software Development	alternative allégée de UP intégrant des préceptes agiles. EssUP est composée d'un ensemble de pratiques, dites "essentielles", qui peuvent être librement sélectionnées et combinées offrant ainsi une grande flexibilité dans son application
DSDM(Dynamic Systems Development Method)	DSDM est une méthode qui combine les principes de développement itératif et incrémental avec une collaboration étroite entre les développeurs et les utilisateurs finaux.

PLAN

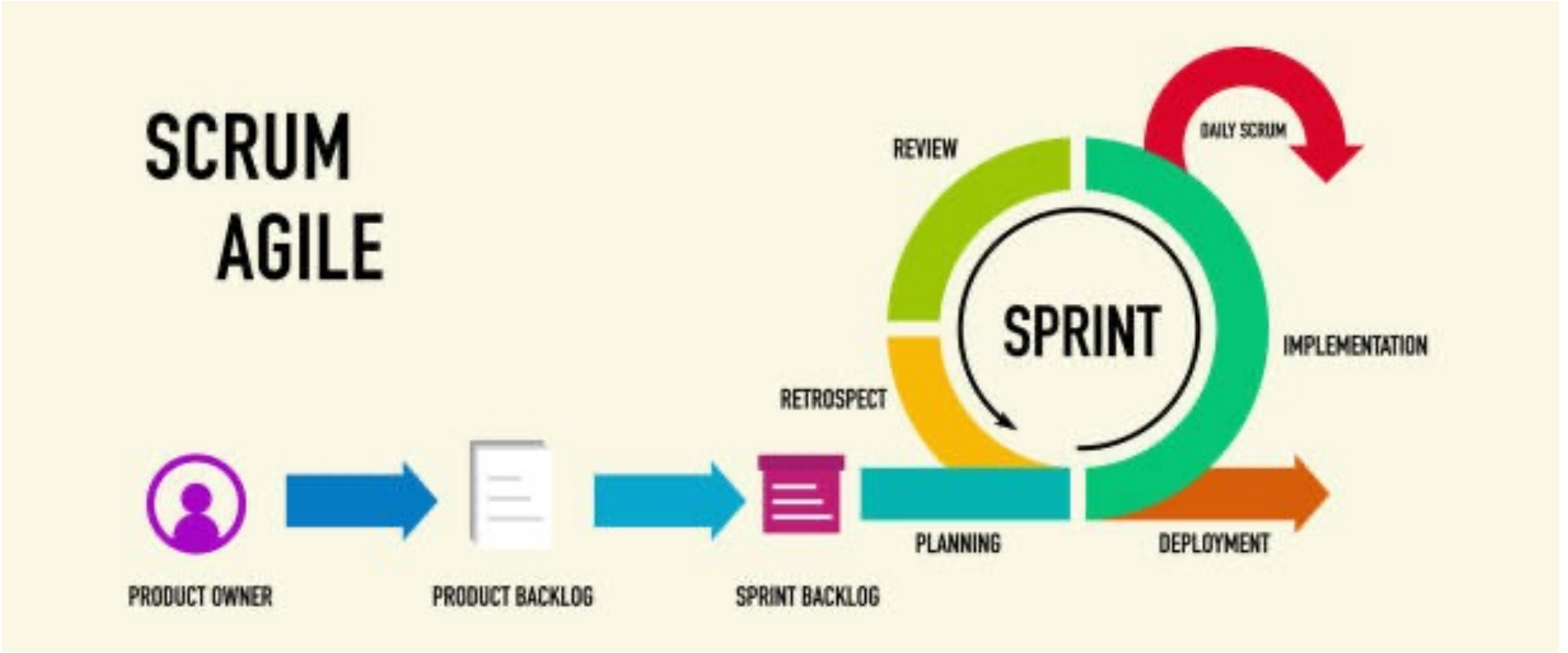
1. INTRODUCTION
2. ACTIVITÉS DU GL
3. MODÈLES DU GL
4. **MÉTHODES DU GL**
5. ARCHITECTURES LOGICIELLES
6. CONCLUSION

Méthode Scrum



Méthode Scrum

- Scrum est l'une des méthodes agiles les plus utilisées.
- Elle se concentre sur des cycles de développement courts appelés sprints, qui durent généralement entre deux et quatre semaines.
- Chaque sprint se termine par une revue du sprint et une rétrospective.


SCRUM
AGILE
REVIEW
DAILY SCRUM
SPRINT
IMPLEMENTATION
RETROSPECT
PLANNING
DEPLOYMENT
PRODUCT OWNER
PRODUCT BACKLOG
SPRINT BACKLOG

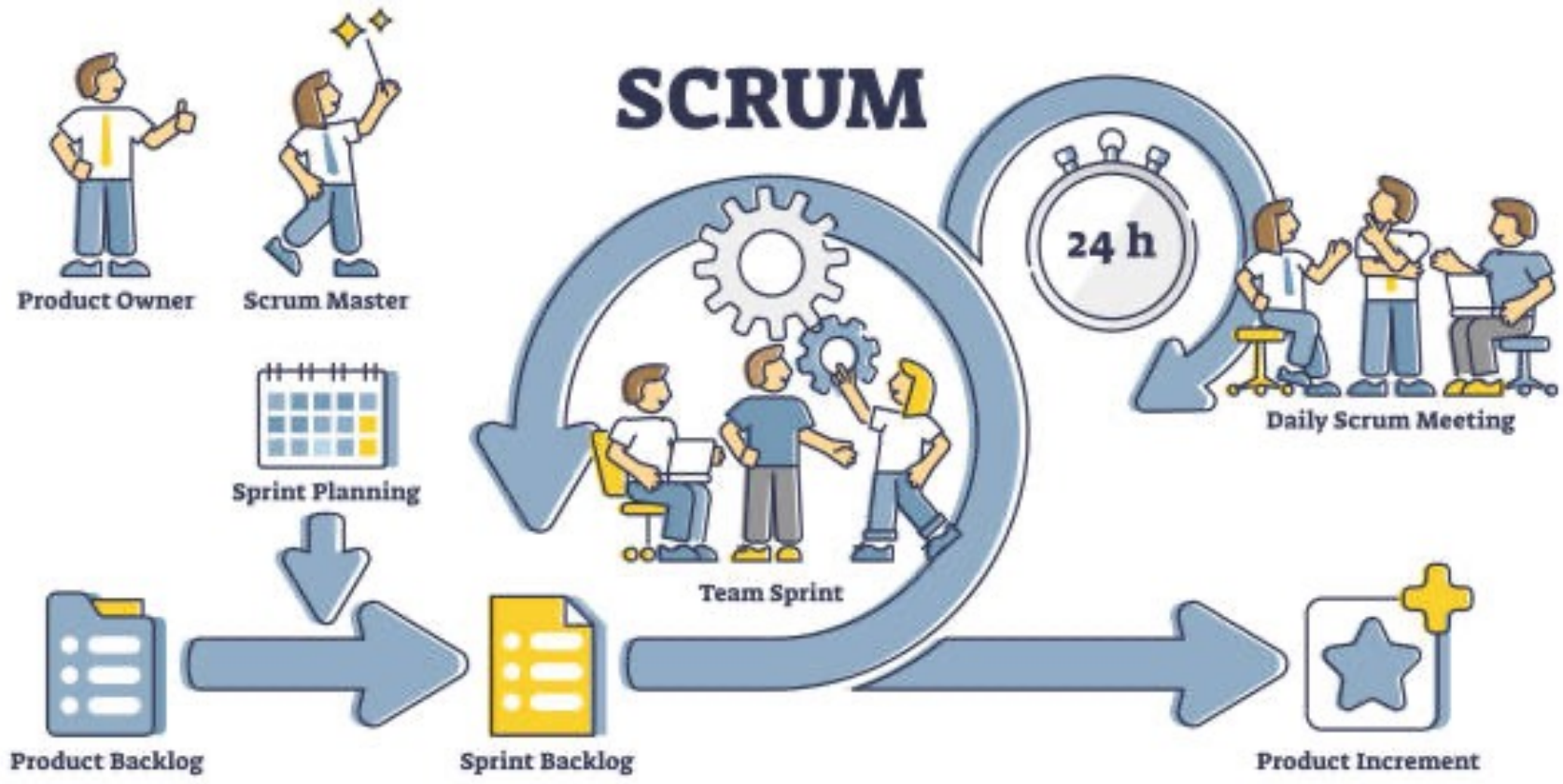
Principes de Scrum

1. **Transparence :** Tous les aspects significatifs du processus doivent être visibles pour ceux qui sont responsables du résultat.
2. **Inspection :** Les utilisateurs de Scrum doivent fréquemment inspecter les artefacts Scrum et les progrès vers un objectif de Sprint pour détecter des variances indésirables.
3. **Adaptation :** Si un processus, ou le produit résultant de celui-ci, est en dehors des limites acceptables, et que le produit final sera inacceptable, le processus ou le matériel étant travaillé doit être ajusté.

Artefacts de Scrum

- **Product Backlog (Backlog du Produit) :** Liste ordonnée de tout ce qui pourrait être nécessaire dans le produit.
  - Gérée et priorisée par le Product Owner.
- **Sprint Backlog (Backlog du Sprint) :** Ensemble des éléments du Product Backlog sélectionnés pour le Sprint, plus un plan pour livrer l'incrément de produit et atteindre l'objectif du Sprint.
  - Géré par l'équipe de développement.
- **Increment (Incrément) :** Somme de tous les éléments du Product Backlog complétés durant un Sprint et les incréments des Sprints précédents.
  - Doit être en état utilisable, que le Product Owner décide de le relâcher ou non.

Artefacts de Scrum



Rôles dans Scrum

1.Product Owner (Propriétaire du Produit) :

1. Responsable de maximiser la valeur du produit et du travail de l'équipe de développement.
2. Gère le backlog du produit (Product Backlog), en le priorisant et en s'assurant que l'équipe comprend bien les éléments à travailler.

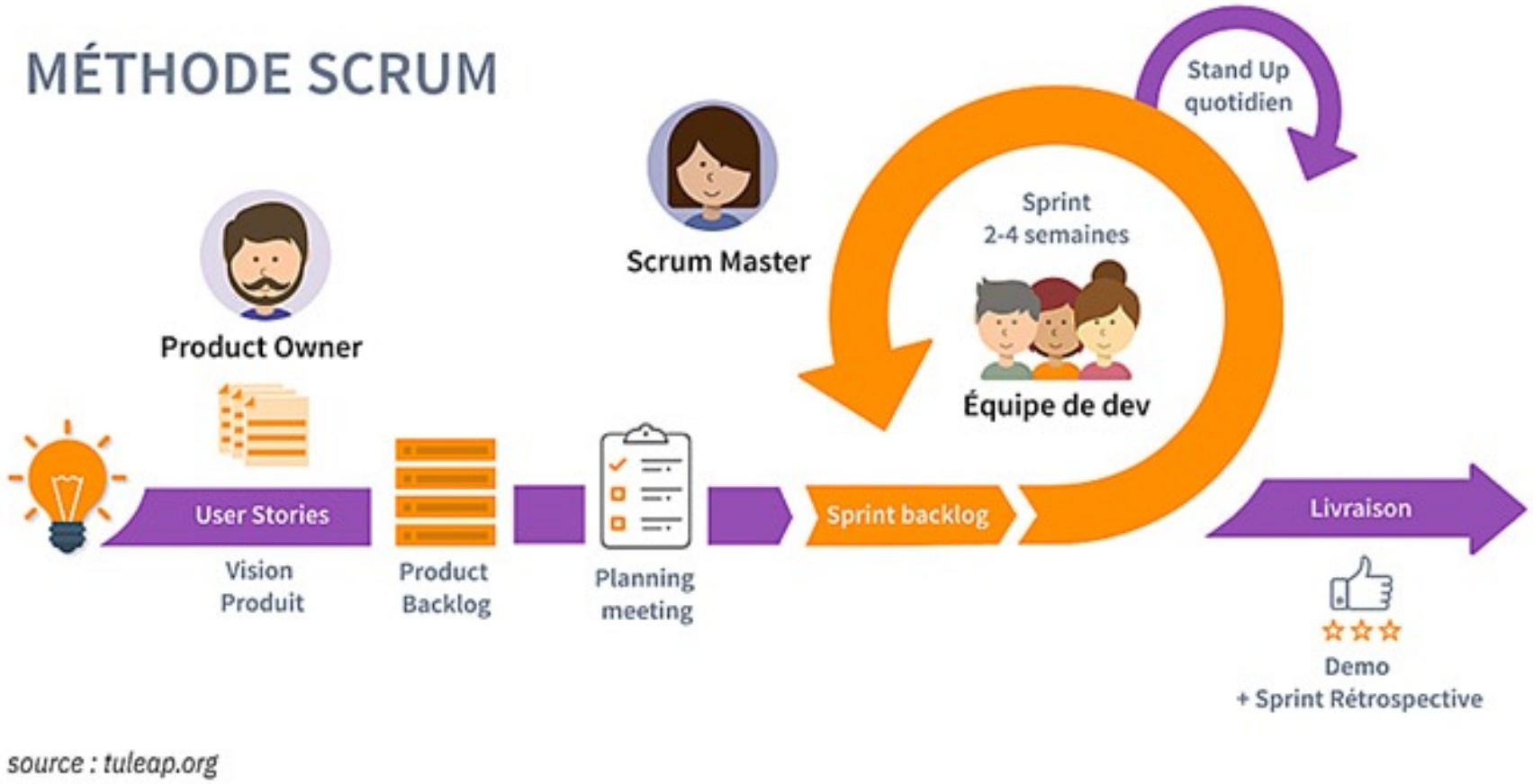
2.Scrum Master :

1. Responsable de faciliter le processus Scrum et de s'assurer que l'équipe suit les pratiques et règles de Scrum.
2. Aide à résoudre les obstacles qui peuvent bloquer l'équipe de développement.
3. Travaille comme un coach pour l'équipe et l'organisation, pour améliorer la productivité et l'efficacité.

3.Équipe de Développement :

1. Composée de professionnels qui font le travail de livraison d'un incrément de produit potentiellement livrable à la fin de chaque Sprint.
2. Auto-organisée et multi-disciplinaire, possédant toutes les compétences nécessaires pour créer un produit.

Rôles dans Scrum



Événements de Scrum

1. Sprint :

1. Période fixe de 2 à 4 semaines durant laquelle un incrément de produit utilisable est créé.
2. Commence immédiatement après la conclusion du sprint précédent.

2. Sprint Planning (Planification du Sprint) :

1. Réunion pour planifier le travail à accomplir durant le sprint.
2. L'équipe de développement sélectionne les éléments du Product Backlog à compléter durant le Sprint.
3. Définition d'un objectif de Sprint qui fournit une direction pour le sprint.

3. Daily Scrum (Scrum Quotidien) :

1. Réunion de 15 minutes tenue chaque jour pour synchroniser les activités et créer un plan pour les prochaines 24 heures.
2. Les membres de l'équipe discutent de ce qu'ils ont fait depuis la dernière réunion, de ce qu'ils feront d'ici la prochaine réunion, et des obstacles éventuels.

4. Sprint Review (Revue du Sprint) :

1. Réunion tenue à la fin du sprint pour inspecter l'incrément et adapter le Product Backlog si nécessaire.
2. L'équipe de développement présente le travail accompli à toutes les parties prenantes.

5. Sprint Retrospective (Rétrospective du Sprint) :

1. Réunion tenue après la revue du sprint et avant la planification du prochain sprint.
2. L'équipe réfléchit à la manière dont le sprint s'est déroulé et identifie des actions spécifiques à améliorer pour les prochains sprints.

Événements de Scrum



Avantages de Scrum

- **Flexibilité :** Permet des ajustements rapides en réponse aux changements de priorités et aux retours des clients.
- **Transparence et Collaboration :** Favorise une communication ouverte et une collaboration étroite entre tous les membres de l'équipe et les parties prenantes.
- **Livraisons Régulières :** Les sprints courts permettent des livraisons fréquentes de nouvelles fonctionnalités.
- **Amélioration Continue :** Les rétrospectives de sprint permettent à l'équipe de réfléchir et de s'améliorer en continu.

Inconvénients de Scrum

- **Nécessite un Engagement Fort :** Demande un engagement constant de la part de tous les membres de l'équipe et des parties prenantes.
- **Peut être Difficile à Mettre en Œuvre :** Peut nécessiter une formation et une adaptation culturelles pour être mis en œuvre efficacement.
- **Documentation Moins Formelle :** Moins de documentation formelle peut poser des problèmes de maintenance et de conformité dans certains contextes.

Résumé de Scrum

- Scrum est une méthodologie puissante pour gérer des projets de développement logiciel dans des environnements dynamiques et incertains.
- Sa focalisation sur l'itération, la collaboration et l'amélioration continue permet aux équipes de rester flexibles et réactives, tout en maintenant un haut niveau de qualité et de productivité.

PLAN

1. INTRODUCTION
2. ACTIVITÉS DU GL
3. MODÈLES DU GL
4. METHODES DU GL
5. ARCHITECTURES LOGICIELLES
6. CONCLUSION